

Solution Requirements

Date	1 October 2026
Team ID	SWTID-2026-4683
Project Name	EduGenie: Google Gemini Powered Learning Assistant
Maximum Marks	4 Marks
Submitted by	Dechen

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what the EduGenie solution will do and how well it will perform. Functional requirements describe system behaviors and features; non-functional requirements describe quality attributes such as performance, security, usability, and reliability.

The requirements below are based on the EduGenie project description, scenarios, modules, API endpoints, frontend workflow, deployment approach, and stated future enhancements. Where the project document does not specify a numeric target, the acceptance criterion is kept qualitative rather than inventing a measured value.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Question & Answer	The system shall accept a learner's academic or general-knowledge question and return an AI-generated answer through the QnA functionality.	High
2	Concept Explanation	The system shall explain complex topics in simplified, concise and readable language using the explanation module.	High
3	Quiz Generation	The system shall generate three multiple-choice questions from a supplied passage/topic, with four options and a correct answer for each question.	High
4	Summarization	The system shall summarize long educational passages into concise, easy-to-understand content while retaining core information.	High
5	Learning Path	The system shall generate a structured learning path from beginner to advanced concepts, with suggested learning resources and guidance.	High
6	Frontend Interface	The web interface shall provide a task dropdown, text input area, submit button, and a result area for the selected learning task.	High
7	Backend APIs	FastAPI shall expose endpoints for /qa, /explain, /quiz, /summarize, and /learn/recommendations and connect each endpoint to its module.	High
8	AI Integration	The solution shall use Gemini for Q&A; quiz generation, summarization and learning recommendations, and LaMini-Flan-T5 for concept explanation.	High

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
9	Real-Time Result	The system shall send the form submission to the FastAPI backend and display the returned result below the input area.	High
10	Local Deployment	The solution shall be runnable locally using Uvicorn with the documented command and accessible through the local web address.	Medium
11	Error Handling	The AI modules shall handle invalid or incomplete model responses and return a useful error message where generation or parsing fails.	Medium
12	Responsive UI	The frontend shall provide responsive styling for input areas, buttons and result containers.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The application should process learner requests efficiently and display generated results without unnecessary delay.	Result is returned and displayed successfully after request processing.
2	Scalability	The architecture should allow the modular FastAPI backend and cloud-based AI integration to be extended as usage grows.	Modules and endpoints remain separable and can be extended without redesigning the whole application.
3	Security & Data Privacy	API credentials and configuration should be handled securely; sensitive keys should not be exposed in the frontend or source code.	Gemini API key is stored securely and not exposed in the user interface.
4	Reliability & Availability	The system should provide consistent responses and useful error handling when an AI response is invalid or incomplete.	Errors are handled gracefully and a meaningful error message is returned when required.
5	Usability & Accessibility	The interface should remain simple, readable and easy to use for learners with different levels of technical experience.	A learner can select a task, enter content, submit it and understand the returned result.
6	Maintainability	The modular architecture should make it easy to update explanation, QnA, quiz, summary and learning-path functionality independently.	Each major feature has a separate module and corresponding FastAPI endpoint.
7	Portability	The application should run in a standard Python environment and be usable on devices with modest local resources while leveraging cloud AI where required.	Application runs using the documented Python/FastAPI/Uvicorn setup.

Requirements Traceability / Source Notes

Project capabilities used: question answering, simplified explanations, quiz generation, personalized learning recommendations and summarization. The project uses FastAPI with an HTML/CSS frontend, Gemini-based cloud functionality and LaMini-Flan-T5 for concept explanation.

Documented API endpoints: /qa, /explain, /quiz, /summarize, /learn/recommendations.

Documented deployment: local execution with Uvicorn using uvicorn main:app --reload and access through the local application address.

Proposed vs. documented requirements

The functional requirements above are directly derived from the project's described features and workflow. Because the project document does not define numeric service-level targets such as response-time limits, concurrent-user counts, uptime percentages, or formal compliance standards, the NFR acceptance criteria intentionally use observable qualitative conditions rather than inventing unsupported numbers.

Submitted by: Dechen
Team ID: SWTID-2026-4683
Project: EduGenie: Google Gemini Powered Learning Assistant